



Functions and Scope

Task

[Visit our website](#)

Introduction

Functions are a fundamental part of programming that allow you to organise your code into reusable blocks, each designed to perform a specific task. Utilising functions helps break down complex problems into manageable parts, and enhances code readability and maintainability. Functions can be user-defined or built-in, with built-in functions being an integral part of the JavaScript language and immediately available for use.

Understanding scope is essential, as it defines where variables can be accessed and manipulated within your code. Functions create local scopes that limit variable access to specific sections, reducing naming conflicts and ensuring that code components interact as intended. Mastery of scope leads to well-organised and more secure code, facilitating smoother development and debugging processes.

Basic JavaScript functions

Declaring a function in JavaScript involves providing a function name, followed by a list of parameters enclosed in parentheses `()`, and the function body enclosed within curly braces `{}`. Here's the basic syntax:

```
function functionName(parameter1, parameter2, ...parameterN) {  
    // Function body  
    // Statements that define what the function does when called  
}
```

Here's a basic example of a function in JavaScript:

```
function greet() {  
    console.log('Hello, World!');  
}  
  
greet();
```

In this case, **greet** is a function that prints out “Hello, World!” to the console. You can **call** this function using its name followed by parentheses `()`, like this: **greet()**. We'll touch further on how to call this function later in this document.

JavaScript function key components

A JavaScript function has three key components:

- **Parameters:** These are variables listed as a part of the function definition. They act as placeholders for the values on which the function operates, known as arguments.
- **Function body:** Enclosed between curly braces {}, the function body consists of statements that define what the function does.
- **Return statement:** How a function sends the result of its operations back to the caller. Not all functions have to return a value; those that don't are often used for their side effects, such as modifying the global state or producing an output.

```
// In the function below num1 and num2 are parameters  
function addNumbers(num1, num2) {  
  let sum = num1 + num2; // Function body  
  return sum; // Return statement  
}  
  
let result = addNumbers(5, 3); // Calls the function with five and three  
console.log(result); // Output the result of the addition
```

In the above example, **num1** and **num2** are parameters, which act as placeholders for values that will be passed into the function when it is called. The lines of code within {} constitute the function body where the logic of the function is defined.

The **return** statement returns the value of **sum**, which is then assigned to the **result** variable. This ensures that the value of **sum** is accessible outside the function. Finally, **console.log(result);** outputs the result to the console.

Calling a function

After a function has been declared, it can be invoked or **called** anywhere in your code by using its name followed by parentheses (). If the function requires parameters, you'll include those within the parentheses. Each argument corresponds to the position of the parameter in the function declaration. To call the function **greet**, we would do this:

```
function greet(name) {  
    console.log(`Hello, ${name}`); // Print greeting based on the name  
    // parameter  
}  
  
greet("Alice"); // Call the greet function to output: Hello, Alice
```

In this case, the string “Alice” is an argument that's passed into the **greet** function. When the function is called, it replaces the **name** parameter with “Alice” and executes the function body.

Here's another example, a variant on the **addNumbers** function above, where we again have a function with two parameters:

```
function addNumbers(num1, num2) {  
    console.log(num1 + num2); // Log the sum of num1 and num2 to the console  
}  
  
// Calling the function with five and 10 passed in as arguments  
  
addNumbers(5, 10); // Output: 15
```

In this example, **addNumbers** is a function that takes two parameters, **num1** and **num2**. When we call **addNumbers(5, 10)**, the function takes those arguments, adds them together, and outputs the resulting value to the console.

Understanding how to declare and call a function is a key part of mastering JavaScript, enabling you to write modular, reusable code that keeps your programs **DRY** (Don't Repeat Yourself) and efficient.

First-class functions

JavaScript treats functions as first-class objects, meaning that functions can be treated like any other value. This provides several powerful capabilities in programming, such as passing functions as arguments to other functions, returning functions from functions, and assigning functions to variables. Understanding this concept is also key to mastering JavaScript's functional programming capabilities.

Assigning functions to variables

You can assign a function to a variable, just like any other value in JavaScript. This allows you to treat the function as an object that can be passed around or called later. Below is an instance of how this can be done:

```
const greet = function(name) {  
  console.log(`Hello, ${name}`);  
};  
greet("Alice"); // Output: Hello, Alice
```

In this example, a function that logs a greeting is assigned to the variable **greet**, and then called just like any function.

Passing functions as arguments

In JavaScript, functions are first-class objects, meaning they can be passed as arguments to other functions. This allows for more flexible and reusable code, where behaviour can be customised by passing different functions as needed.

```
function processUserData(userData, callbackFunction) {  
  // Process user data and pass the result to the callback function  
  const processedData = userData.toUpperCase();  
  callbackFunction(processedData);  
}  
  
function displayData(data) {  
  console.log(data);  
}  
  
processUserData("john doe", displayData); // Output: JOHN DOE
```

Here, the **processUserData** function accepts **userData** and a **callbackFunction** as arguments. After processing the data (converting it to uppercase), it calls the **callbackFunction** with the processed result. This pattern makes the **processUserData** function highly adaptable, as you can provide different callback functions depending on the desired outcome.

First-class functions in practice

Treating functions as first-class citizens enables powerful programming patterns in JavaScript. This flexibility allows you to create modular and maintainable code, especially in large applications where scalability is crucial.

For example, in functional programming, you might pass multiple functions as arguments to higher-order functions, chain them to perform complex operations, or use them to dynamically modify data:

```
const multiplyByTwo = (num) => num * 2;
const subtractFive = (num) => num - 5;
function transformNumber(num, ...operations) {
  return operations.reduce((result, operation) => operation(result), num);
}
console.log(transformNumber(10, multiplyByTwo, subtractFive)); // Output: 15
```

Here, the **transformNumber** function accepts a number and a list of operations to perform. Each function is applied in sequence, demonstrating how first-class functions enable clean and elegant solutions to complex problems.

Integrating first-class functions with basic functions

Functions are integral to JavaScript's design, and understanding how first-class functions work opens up new ways to structure your code. Functions can be passed as arguments, returned from other functions, and assigned to variables, which adds an extra level of flexibility and reusability.

For example, consider using first-class functions to simplify code, where you pass different functions as arguments based on different use cases:

```
// Higher-order function that accepts a function as an argument
function operate(a, b, operation) {
  return operation(a, b);
}
const sum = function(x, y) {
  return x + y;
}
```

```
};

const multiply = function(x, y) {
  return x * y;
};

console.log(operate(5, 10, sum)); // Output: 15
console.log(operate(5, 10, multiply)); // Output: 50
```

Here, **operate** is a higher-order function that accepts two numbers and a function (**sum** or **multiply**). This flexibility lets you change the behaviour of **operate** by simply passing different functions as arguments, demonstrating the power of first-class functions.

Parameter versus arguments

The primary difference between parameters and arguments lies in where they show up in the code:

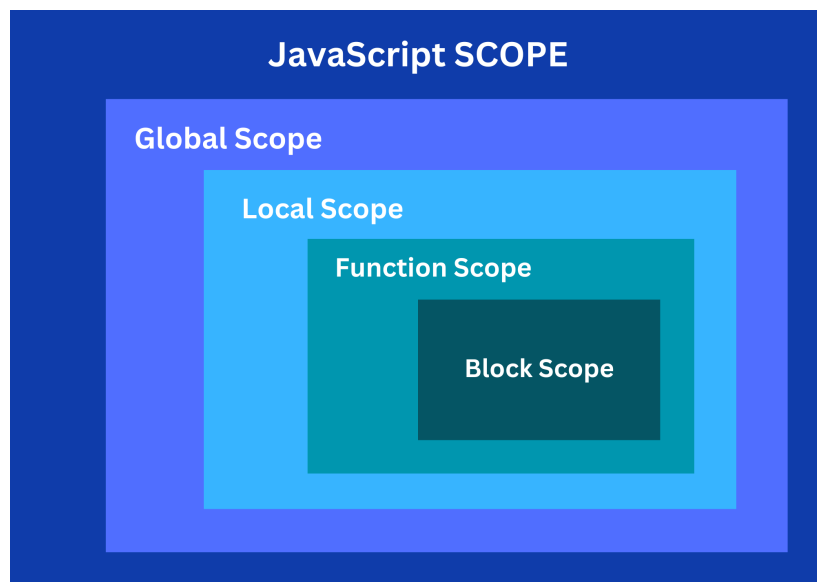
Parameters	Arguments
Parameters are used when defining a function. They represent the 'input' the function needs to do its job, and they act as placeholders for actual data.	Arguments are used when calling a function. They represent the actual 'input' that will be operated on by the function's code.

Scope

Scope is what we call a program's ability to find and use variables in a program. Another way to look at it is that the scope of a variable determines where in the code it can be seen from. The rule of thumb is that a function is covered in one-way glass: it can see out, but no one can see in. This means that a function can call variables that are 'outside' the function, but the rest of the code cannot call variables that are defined 'inside' the function.

Types of scope

JavaScript primarily has three types of scope: global scope, function scope, and block scope. Function and block scopes both fall into the category of local scope. Let's look at each in more detail:



Global scope

When a variable is declared outside all functions or block scopes, its scope is global. Global variables can be accessed from any part of the code, whether within a function or outside.

```
let globalVar = "I am global!"; // This is a global variable

function testScope() {
  console.log(globalVar); // Output: I am global!
}

testScope();
```

Function scope

Variables declared within a function are accessible only within the function body and are said to have the function scope. They cannot be accessed outside of the function in which they are declared. Attempting to access a function-scoped variable from outside the function will result in a reference error.

```
function testScope() {
  // Defined as a local variable (only accessible from inside the testScope function)

  let localVar = "I am local!";
```



```
    console.log(localVar); // Outputs: I am local!
  }
testScope();

console.log(localVar); // Outputs: Uncaught ReferenceError: localVar is not defined
```

Block scope

The introduction of the **let** and **const** keywords in ES6 introduced block scope into JavaScript. Variables declared with **let** or **const** are confined to the block in which they are declared (unlike variables declared with **var**, which are function-scoped). Attempting to access block-scoped variables outside their block results in a reference error, as they are only accessible within the block where they were defined.

```
if (true) {
  // blockVar is only accessible from inside this code block

  let blockVar = "I am block scoped";

  console.log(blockVar); // Outputs: I am block scoped
}

console.log(blockVar); // Outputs: Uncaught ReferenceError: blockVar is not defined
```

Nested functions

A nested function is a function defined inside another function. This allows the inner function to access variables and parameters of the outer function. Nested functions help organise code and keep related functionality together, making code more modular and maintainable:

```
// The outer function "multiplier" that takes one parameter "x".
function multiplier(x) {

  // The nested function "inside" which takes one parameter "y".
  function inside(y) {
```

```

    // Return the product of "x" and "y".
    return x * y;
}

// Return the "inside" function. It has access to "x" from "multiplier".
return inside;
}

// Example usage:
const multiplyByTwo = multiplier(2);
console.log(multiplyByTwo(5)); // Output: 10

```

In the provided code, **inside** is a nested function within the **multiplier** function. When **multiplier** is called, it doesn't immediately perform a calculation; instead, it returns the **inside** function. This returned nested function can then be used later, allowing you to apply further operations. However, the nested **inside** function is still able to access the parameter **x** and the argument which was passed to the **multiplier** function.

For instance, if you call **multiplier(2)**, the number **2** is passed as an argument and is stored in the parameter **x**. The **multiplier** function would then return the **inside** function, which can use that stored value of **x** whenever it is called later. This means that even when the **multiplier** function is no longer running, the inner function still has access to the value of **x**.



Practical task

1. Create a JavaScript file called **functionScopePractice.js**.
2. Define the following functions and implement the required functionalities listed:
 - **sumArray**:
 - Accepts an array of integers as the parameter.
 - Uses a local variable to store the sum of the integers.
 - Calculates the total sum using a loop or the **reduce()** method.
 - Returns the total sum of the integers.

- **variableScopeTest:**
 - Defines a variable inside the function body using **let** or **const**.
 - Logs the variable inside and outside of the function.
 - Includes comments explaining the observed behaviour for each **console.log()**.
- **outerFunction:**
 - Declares a local variable and defines a nested function (**innerFunction**) that accesses the variable.
 - Calls **innerFunction** from within **outerFunction**.
- **demonstrateGlobalVsLocal:**
 - Declares a global variable (outside the function) and a local variable (inside the function) with the same name, but with different values assigned to each.
 - Logs the global variable outside the function
 - Logs the local variable inside the function.

3. Test the functions:

- Create an array with at least four integers.
- Call **sumArray** using the array (passed in as an argument) and log the result.
- Call **variableScopeTest**.
- Call **outerFunction** and ensure the nested function accesses the variable as expected.
- Call **demonstrateGlobalVsLocal** and observe the logged variables.

4. Handle edge cases:

- Ensure **sumArray** handles an empty array by returning **0**.

Place the **functionScopePractice.js** file in your task folder and submit it.

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
